

Supply-Chain Disruption Triage with Responsible Multi-Agent AI

Shivendra | Track A — Agentic / Multi-Agent AI

1. Problem and domain

I chose S1: Supply Chain Risk and Optimization. A supplier delay is rarely an isolated event: its significance depends on current inventory, demand uncertainty, alternate stock, and the cost of acting too soon or too late. That makes it a real AI opportunity and a domain where careless automation is expensive. A system that expedites every late shipment wastes money; one that overlooks a genuine shortage stops production.

My goal is not to automate purchasing. It is to shorten the path from a disruption signal to a justified, reviewable recommendation. Agentic AI suits this because the work is a sequence of bounded tasks — retrieve evidence, estimate impact, apply operating policy, escalate — but in a sensitive operational setting that autonomy has to be earned through controls rather than assumed.

2. Approach and algorithm decisions

Two failure modes bracket this problem. Hand the whole task to an LLM and it will confidently invent a purchase order. Hardcode the whole task and you have a rules engine wearing an agent costume. The design decision this submission is actually about is where to draw the line between them.

Concern	Owner	Rationale
Which evidence to gather, in what order, with what lookback	GPT planner	Genuine judgement under uncertainty. Each agent's tool allow-list is wider than its minimum requirement, so tool selection is a real decision.
Whether a proposed tool call may run at all	Guardrail layer	Model intent is untrusted input until validated against allow-list, schema, value bounds, investigation scope, and a duplicate-call guard.
Every number: cover, gap, quantity, risk level	Policy engine	Reproducible and challengeable by a planner. No figure originates in model text.
Whether anything happens	Human	A LangGraph interrupt before the action node. Approvals carry a planner identity.
Explaining the result	GPT narrator	Fluency helps a planner, so every numeric token in the explanation is checked against the computed facts before it enters the trace.

The consequence is that the model can be wrong about *how to investigate* — recoverable, and visible in the trace — and cannot be wrong about *what to do*. I chose LangGraph over CrewAI and AutoGen because this workflow needs visible state transitions, checkpoints, conditional routing, and a first-class interrupt immediately before an operational action. CrewAI is faster for role-based prototypes but less precise for approval gates; AutoGen's open-ended dialogue is a liability when a workflow must fail safely. The trade-off is more explicit graph code in exchange for predictable behaviour.

For the forecast I used a transparent mean over a model-chosen lookback with a dispersion signal, and rejected a heavier model: synthetic data cannot justify claims of superior accuracy, and a simple baseline is easy for a planner to challenge and replace once historical data exists. The more consequential modelling choice was to measure exposure against *safety stock* rather than bare on-hand. Days of cover alone reports a healthy position for a SKU already below its contractual floor.

3. Results and error analysis

The evaluation is scenario-based rather than a misleading accuracy benchmark. Eight cases run against both a scripted reference plan and a live model, so a regression can be attributed to the graph or to the model. Beyond its own expected outcome, every case is checked against invariants that must hold universally: the proposal was blocked at creation, no action event precedes the human decision, the approval is attributed, and quantity respects both real surplus and the policy cap.

Metric	Result	Why it is the metric
Missed escalations	0 / 8	Proceeded when it should have stopped. The only class that causes direct operational harm.
False escalations	0 / 8	Stopped on a normal operating case. The mechanism by which an assistive system gets switched off.
Actions without approval	0 / 8	The human gate is either real or decoration.
Proposals within policy bounds	8 / 8	Quantity $\leq$ real transferable surplus and $\leq$ the named single-proposal cap; rollback and evidence chain present.
Guardrail blocks on the adversarial case	1 / 1	An out-of-scope tool call is refused before the query reaches the database.
Ungrounded narratives	0	No model explanation introduced a figure the policy engine never computed.

Two of those cases exist because of a defect I found in my own first version. It detected contradictory evidence by comparing statuses across returned rows, which meant any SKU with two open purchase orders looked contradictory. It passed its tests because the fixture contained exactly one PO per SKU. In production it would have escalated nearly everything and been switched off within a week. Contradiction is now defined *within* a purchase order, between updates sharing a reported date, and two scenarios pin the distinction: several open POs, and a later update correcting an earlier one, must both proceed.

The dominant failure mode remains silent confidence in bad data, so the system fails closed on absent, unverified, or contradictory evidence, on insufficient history, on tool failure, and on guardrail violation — each with a distinct machine-readable escalation type, so an audit consumer can tell a refused proposal from a data failure without parsing prose. It can still fail if verified source data is stale, if demand shifts abruptly, or if alternate inventory is reserved elsewhere. Those are surfaced as assumptions in the trace rather than hidden behind a fluent explanation.

4. Production and limitations

In production I would keep read-only analytics strictly separate from any ERP write path, use least-privilege credentials, and hold durable workflow checkpoints rather than the demo's in-memory store. Monitoring would cover data freshness, tool failure rate, guardrail violations by violation code, escalation rate split by escalation type, planner approval and override rates, narrative grounding failures, latency per node, and model cost per triage. New policy or model versions would be replayed against historical cases and released through a canary.

The central limitation is the synthetic dataset. It demonstrates that the workflow is reproducible and that the safety controls behave as intended; it does not demonstrate business impact, and this evaluation makes no forecast-accuracy claim. Whether a given transfer was commercially right depends on transport cost, allocation priority, and planner knowledge that is not modelled here. Before deployment I would run the system in shadow mode against historical ERP outcomes with planner adjudication, calibrate thresholds against real demand by SKU and site, and only then widen the scope of recommendations.